

DAY-4

POSTMAN

- **Postman :**

Postman is a software (application) used to **test APIs**.

(or)

Postman is a tool that allows us to send API requests to a server and view the server's response. **Example:**

Customer → Waiter → Kitchen

Customer = Client

Waiter = API

Kitchen = Server

Food = Response

You never go directly into the kitchen.

Similarly,

Your application never directly goes to the database.

It always uses APIs

Why do we use Postman?

- To test APIs.
- To check whether APIs are working correctly.
- To send requests to the server.
- To receive responses from the server.
- To debug backend APIs before connecting the frontend.

-
- **Why do we install Postman?**

Suppose you are developing a project.

Normally:

Frontend → Backend → Database

If you want to check whether the backend API is working, you don't have to build the frontend first.

Instead:

Postman → Backend API → Database

Postman directly communicates with the backend.

- **How to install Postman (Windows 11) Step 1:**

Open any browser (Chrome, Edge, etc.).

Step 2:

Search for **Postman**.

Step 3:

Open the official Postman website.

Step 4:

Click **Download**.

Step 5:

Choose **Windows 64-bit** (if your laptop is Windows 11).

Step 6:

Wait for the download to finish.

Step 7:

Open the downloaded .exe file.

Step 8:

Follow the installation instructions.

Step 9:

After installation, open Postman.

• Simple Flow of How Postman Works

Choose Method (GET/POST/PUT...)



Enter URL (API Endpoint)



Add Params/Headers/Body (if required)



Click Send



Server processes the request



Response appears in the Response section.

• HTTP Method Dropdown

1. GET
2. POST
3. PUT
4. PATCH
5. DELETE
6. HEAD
7. OPTIONS

These are called **HTTP Methods**.

Each method has a different purpose.

1.GET

Used to **read/fetch data**.

It doesn't send new data.

No body is required.

If you need to send values, you usually use **query parameters**, not a body.

Example:

Get all employees.

2.POST

Used to **create new data**.

Example:

Add a new employee.

POST request has two important parts:

Header

Header contains extra information about the request.

Example:

Content-Type: application/json This

tells the server:

"The data I'm sending is in JSON format."

Body

The Body contains the actual data.

Example:

```
{  
  "name": "Sindhu",  
  "age": 22  
}
```

This is the information the server stores.

3.PUT

Used to **replace the entire existing record**.

4.PATCH

Used to **update only a few fields**.

5.DELETE

Used to **delete data**.

6.HEAD

Returns only the response headers, not the body.

Mostly used for checking information about a resource.

7.OPTIONS

Shows which HTTP methods the server supports.

Example:

A server may allow:

- GET
 - POST
 - DELETE
-

- **API:** Application Programming Interface

An API is a software interface that enables communication between two applications by sending requests and receiving responses.

Why do we need an API?

Suppose you have:

- A Mobile App (Frontend)
- A Database (Backend)

Can the mobile app directly access the database?

No.

The API receives the request from the client, processes it, interacts with the database if needed, and sends the response back.

Instead, the communication happens like this:

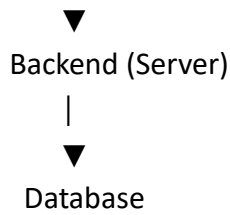
Frontend (Client)

|



API

|



- **What is an Endpoint?**

An **Endpoint** is a **specific URL (address)** through which a client sends a request to an API to perform a particular operation

(or)

An Endpoint is the address of an API where requests are sent.

When the client wants some information, it sends the request to a particular URL.

That URL is called the **Endpoint**.

Example: `http://localhost:8080/students`

Here,

- `http://` → Protocol
- `localhost:8080` → Server Address
- `/students` → Endpoint

The endpoint tells the server **which operation to perform**.

- **What is JSON?**

JSON is one of the input payload

JSON stands for

JavaScript Object Notation

It is the most common format used to send data to the server.

Example:

```
{
  "name": "Sindhu",
  "age": 22
}
```

- **What is Input Payload?**

Payload means:

The actual data sent from the client to the server.

Example:

When registering a student:

Name:

Sindhu

Department: CSE

All this data together is called the **Payload**.

- **Payload Formats**

A payload can be sent in different formats.

- 1. Text**

Definition:

Text payload contains only **plain text**.

Example Hello

Sindhu

Use:

Used when we want to send only simple text.

- 2. JavaScript**

Definition:

JavaScript payload is data written as a **JavaScript object**.

Example

```
{  
  name: "Sindhu", age:  
  22  
}
```

Use:

Used inside JavaScript programs.

3. HTML

Definition:

HTML payload contains **HTML code**.

Example `<h1>Welcome</h1>`

Use:

Used to create and display web pages.

4. XML

Definition:

XML is an older format used to send data.

Example

```
<student>
  <name>Sindhu</name> </student>
```

Use:

Earlier, many APIs used XML.

Nowadays, it is used less because JSON is simpler.

5. JSON (Most Important)

Definition:

JSON (JavaScript Object Notation) is the **most commonly used format** for sending data in REST APIs.

Example

```
{
  "name": "Sindhu",
  "age": 22
}
```

Use:

Almost all modern applications like:

- Spring Boot
- React
- Android Apps
- Banking Apps
- E-commerce Websites use **JSON** to send and receive data.

-
- **We can't run direct query on database** Reasons:
 - Security
 - Prevent hacking
 - Prevent accidental deletion
 - Validate data
 - Check user permissions
 - Keep business logic in one place Instead:

Frontend



API



Database

This is the correct architecture

DBMS

DBMS (Database Management System):

DBMS is software used to create, store, update, and manage databases.

Examples: MySQL, Oracle.

Database (DB):

A database is a collection of data.

Example: Student records.

- **Difference:**

Database	DBMS
Stores data	Manages data
Collection of records	Software

- **CRUD Operations**

CRUD means:

C - Create → Add new data

Q: INSERT INTO Student VALUES (1,'Sindhusha',20);

R - Read → View data

Q: SELECT * FROM Student;

U - Update → Change data

Q: UPDATE Student

SET Age=21

WHERE RollNo=1;

D - Delete → Remove data

Q: DELETE FROM Student

WHERE RollNo=1;

- **How to Create a Database**

CREATE DATABASE College;

- **How to Create a Table**

CREATE TABLE Student(
RollNo INT,
Name VARCHAR(50),
Age INT
);

- **How to See the Entire Table**

SELECT * FROM Student;

- **How to See One Row**

Example: Show student whose RollNo is 1.

```
SELECT *  
FROM Student  
WHERE RollNo=1;
```

- **How to See One Column**

Example: Show only names.

```
SELECT Name  
FROM Student;
```

- **Difference Between WHERE and HAVING**

WHERE

- Filters individual rows.
- Used before GROUP BY.

Example:

```
SELECT *  
FROM Student  
WHERE Age>20;
```

HAVING

- Filters grouped data.
- Used after GROUP BY.

Example:

```
SELECT Branch,COUNT(*)  
FROM Student  
GROUP BY Branch  
HAVING COUNT(*)>2;
```

- **Difference Between CASE and IF in DBMS**

Both **CASE** and **IF** are used to check conditions and return a result.

CASE

CASE is used in SQL queries.

It can check **multiple conditions**.

It is part of the SQL standard.

IF

IF() is a **MySQL function**.

It checks **only one condition**.

If the condition is true, it returns one value.

If the condition is false, it returns another value.

- **Can we use IF inside CASE?**

No. CASE has its own keywords (**WHEN, THEN, ELSE**), so IF is not needed. Use **CASE** for multiple conditions and **IF()** separately for a simple condition.

- **When Do We Use IF?**

Use IF() when you have only one simple condition.

Example:

```
SELECT Name,  
IF(Age >= 18, 'Eligible', 'Not Eligible') AS Status  
FROM Student;
```

Output

Name	Status
------	--------

Ram	Eligible
-----	----------

Ravi	Not Eligible
------	--------------

Use IF() when there are only two possible results:

- True
 - False
-

- **When Do We Use CASE?**

Use CASE when there are multiple conditions.

Example:

```
SELECT Name,  
CASE  
WHEN Marks >= 90 THEN 'Excellent'  
WHEN Marks >= 75 THEN 'Good'  
WHEN Marks >= 35 THEN 'Pass'  
ELSE 'Fail'  
END AS Result  
FROM Student;
```

Output

Name Result

Ram Excellent

Sita Good

Ravi Pass

Kiran Fail

Here, CASE checks several conditions one after another.

